Datamecum

Modelos fundacionales en visión artificial

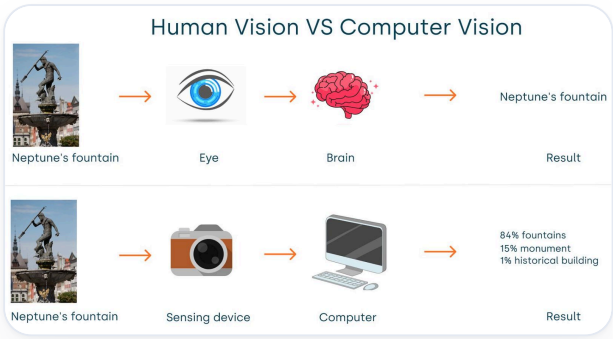
# Tema 1: Detección de objetos

## Día 1 del curso

| | |
|---|---|
| Web del curso | datamecum.sergiovillanuevalopez.com |
| Práctica | Cuaderno 1 en Colab (1_OD.ipynb) |
| Profesor | Sergio Villanueva · Datamecum |

# Qué es la visión artificial

La visión artificial es la disciplina de la inteligencia artificial que busca capacitar a las máquinas para **extraer información significativa y comprender el mundo a partir de imágenes y vídeos digitales**, imitando la percepción humana.




## La brecha semántica: ¿cómo lo ve un humano vs. una máquina?

### Visión humana

Inmediatamente identificamos "un gato naranja durmiendo en un sofá" y comprendemos su contexto.

### Visión computacional

Una máquina percibe una compleja **matriz de millones de píxeles**, cada uno con valores numéricos RGB como [255, 128, 64…].

Las imágenes digitales son matrices de píxeles. La visión artificial aprende representaciones que permiten asociar esos valores con objetos, relaciones o regiones.

# Tareas principales en visión artificial

### Clasificación

Asigna una etiqueta a la imagen entera: dice qué hay, no dónde.

### Detección de objetos

Encierra cada objeto en un recuadro delimitador y le asigna una clase.

### Segmentación

Define el contorno exacto de cada objeto a nivel de píxel.

### Detección de anomalías

Identifica eventos o datos inusuales que no coinciden con un patrón esperado.

### Estimación de pose

Localiza puntos clave (articulaciones) en personas o animales para comprender su postura.

### Otras

OCR, estimación de profundidad, superresolución, keypoint matching…



# ¿Qué salida necesitas?

La misma imagen admite tareas distintas.

### Quieres contar coches y localizar cada uno.

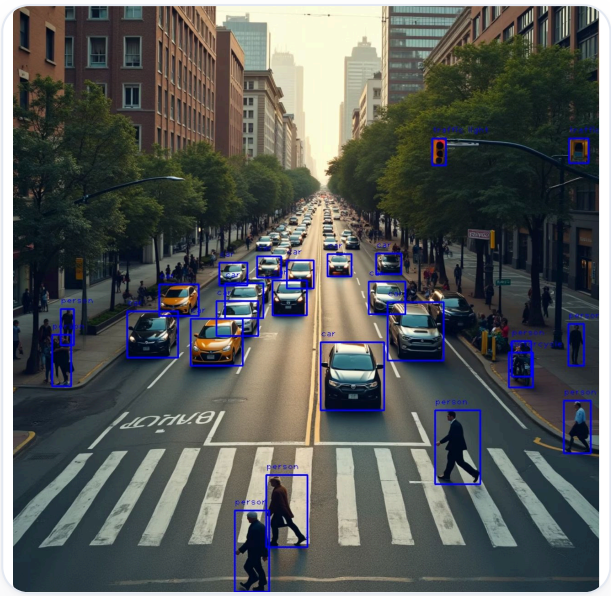
- A Una etiqueta para toda la imagen.
- B Una caja y una categoría por objeto.
- C Una puntuación de confianza para la imagen.

Respuesta B. Exacto: la detección localiza cada instancia y permite contarla.

La clasificación da una etiqueta global. La detección devuelve cajas y categorías por instancia. La segmentación delimita píxeles. Otros problemas incluyen anomalías, pose, OCR, profundidad y superresolución.

# 1. Detección de objetos

La detección de objetos va más allá de la simple clasificación. Su **objetivo principal es localizar y clasificar múltiples objetos simultáneamente** dentro de una imagen, proporcionando para cada uno una caja delimitadora, su categoría y una puntuación de confianza.



**En qué se diferencia de las otras tareas:**

- **Clasificación:** identifica qué hay en la imagen (una etiqueta global).
- **Detección:** localiza cada objeto con cajas y etiquetas múltiples.
- **Segmentación:** delimita el contorno exacto píxel a píxel.
- **Detección de anomalías:** indica si algo en la imagen se sale de lo normal.

# Familias de detectores

Hay muchos detectores. Se agrupan por la red que usan y por cómo sacan las cajas.



# Evolución histórica de los detectores

Tres etapas en diez años.

1 **Two-Stage (2014-2016)**

R-CNN, Fast R-CNN, Faster R-CNN: arquitecturas de dos etapas que primero proponen regiones candidatas y luego las clasifican. Precisos pero complejos y lentos.

2 **One-Stage (2016 →)**

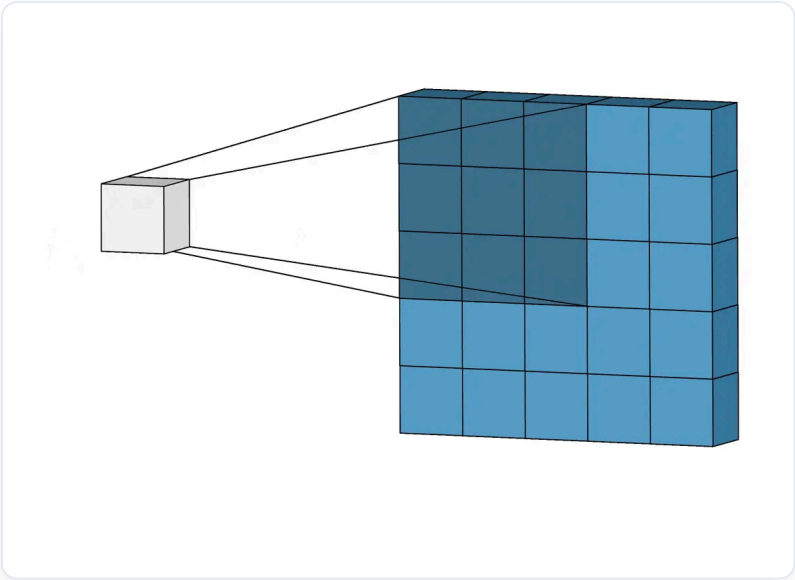
YOLO cambia el planteamiento: una sola red neuronal procesa la imagen completa en tiempo real, prediciendo cajas y clases simultáneamente mediante una cuadrícula.

3 **Transformers (2020 →)**

DETR y RF-DETR: pipelines limpios basados en atención, sin anchors predefinidos ni supresión de no máximos (NMS). Arquitectura más simple y generalizable.

# Redes neuronales convolucionales (CNN)

Las redes neuronales convolucionales (CNN) son la base de casi todos los detectores anteriores a 2020 y siguen dentro de muchos actuales. Procesan la imagen directamente con capas que detectan, de forma jerárquica, bordes, texturas y patrones cada vez más complejos.



Al apilar capas, la red pasa de detectar bordes a reconocer partes de objetos y objetos enteros, sin que nadie programe esas características a mano.

Las CNN incorporan localidad e invariancia aproximada a traslaciones. Siguen dentro de muchos detectores actuales, y una CNN también capta contexto global si se le añaden capas de atención o un campo receptivo grande.

# Capa convolucional

Un filtro de 3 × 3 recorre la imagen: cada píxel de salida es la suma del píxel y sus ocho vecinos, cada uno multiplicado por su peso. Cambia el filtro y mira qué se enciende; toca o pasa el ratón por las imágenes para ver la cuenta.



Entrada, 192 × 108 píxeles



Mapa de características

| Vecindario | | | | Pesos | | | | Salida |
|---|---|---|---|---|---|---|---|---|
| 17 | 47 | 122 | × | −1 | 0 | 1 | = | **419** |
| 18 | 51 | 124 | | −2 | 0 | 2 | | |
| 20 | 51 | 122 | | −1 | 0 | 1 | | |

Se encienden los cambios de luz de izquierda a derecha: las aristas verticales de los edificios. El cielo, liso, queda negro. Un número negativo también se pinta claro: es el mismo borde, de claro a oscuro.

Filtros escritos a mano y calculados en tu navegador. Una CNN aprende sus pesos durante el entrenamiento, y los de su primera capa acaban pareciéndose a estos.

Esta página recoge el filtro de bordes verticales: pesos negativos a la izquierda y positivos a la derecha, de modo que la suma es grande donde la luz cambia en horizontal y cero en las zonas lisas. En el punto marcado, sobre una arista del edificio de la izquierda, la salida vale 419. El filtro de bordes horizontales es el mismo girado 90 grados y enciende cornisas y filas de ventanas. El desenfoque promedia los vecinos y el realce resta los vecinos al centro. Una capa convolucional aplica decenas de filtros como estos a la vez, y las capas siguientes combinan sus mapas para detectar texturas, partes y objetos.

# Detectores de dos etapas

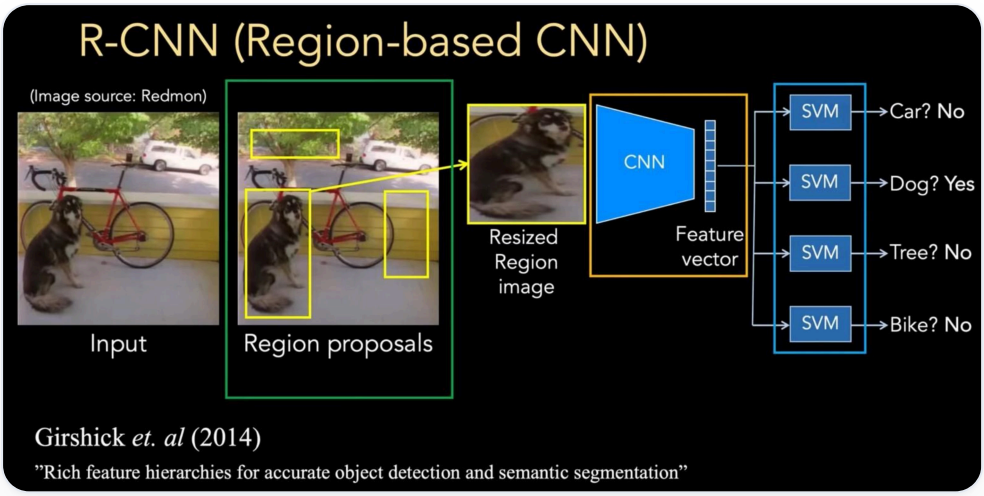
Primero buscan dónde puede haber un objeto; después miran cada zona y deciden qué es.




- **R-CNN (2014):** unas 2.000 zonas por imagen y la CNN pasa por cada una. Muy lento.
- **Fast y Faster R-CNN (2015):** la CNN se pasa una sola vez y las zonas las propone otra red.
- Precisos, pero lejos del tiempo real.

# R-CNN (2014)

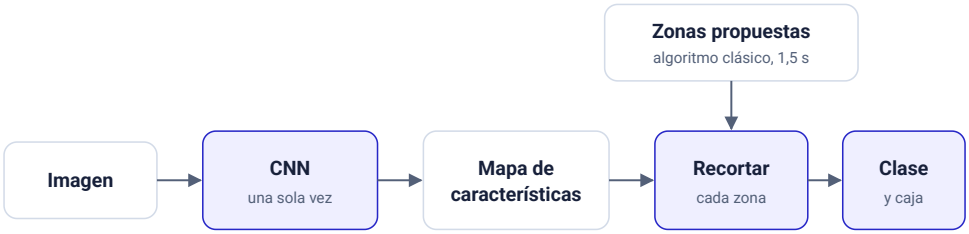
Unas 2.000 zonas por imagen, y la CNN pasa por cada una. Muy lento.




Girshick et al. (2014), "Rich feature hierarchies for accurate object detection and semantic segmentation".

# Fast R-CNN (2015)

La CNN pasa una sola vez por la imagen entera, y cada zona se recorta sobre lo que sale.



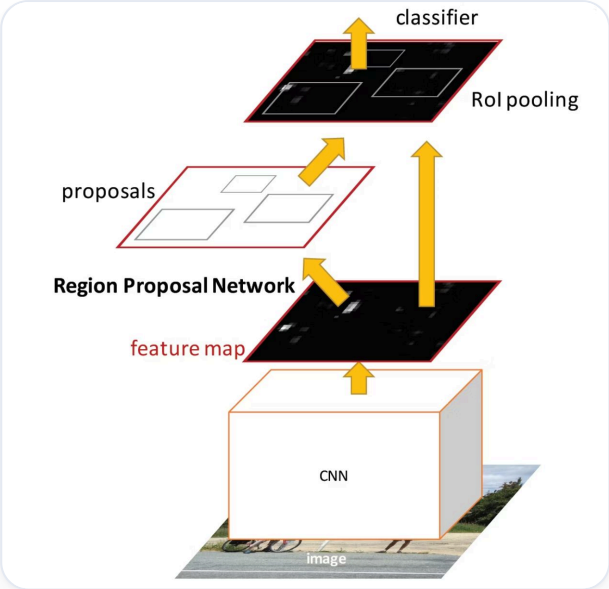

- R-CNN pasaba la CNN 2.000 veces; Fast R-CNN, una.
- Clase y caja se aprenden juntas, en la misma red.
- Lo lento sigue siendo proponer las zonas: un segundo y medio por imagen.

# Faster R-CNN (2015)

Las zonas ya no las propone un algoritmo clásico, sino una red:

- Una CNN saca las características de la imagen.
- Una red pequeña propone zonas donde puede haber objetos.
- Para cada zona, un clasificador da la clase y ajusta la caja.

Muy preciso, pero lento para vídeo.



# Velocidad de la familia R-CNN

**Results**

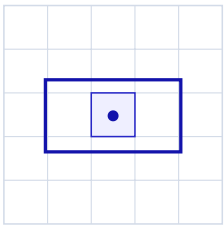
| | RCNN | Fast RCNN | Faster RCNN |
|---|---|---|---|
| **Proposal Generation Method** | Selective Search | Selective Search | RPN |
| **Proposal Generation Time** | 1500 ms | 1500 ms | 10 ms |
| **Predictions for Generated Proposals Time / Image** | 47000 ms | 320 ms | 190 ms |
| **Speedup Factor** | X | 25X | 250X |

El salto está en la generación de propuestas: Selective Search tarda 1500 ms y la RPN de Faster R-CNN, 10 ms.

La tabla compara tres generaciones. Método de propuestas: Selective Search en RCNN y Fast RCNN, RPN en Faster RCNN. Tiempo de generación: 1500 ms, 1500 ms y 10 ms. Predicción sobre las propuestas por imagen: 47000 ms, 320 ms y 190 ms. Factor de aceleración: 1×, 25× y 250×.

# YOLO: una sola pasada

La red mira la imagen una vez y devuelve todas las cajas a la vez, sin buscar zonas antes.



**Cuadrícula sobre la imagen**

La celda donde cae el centro del objeto predice su caja, con su confianza y su clase. Todas a la vez.

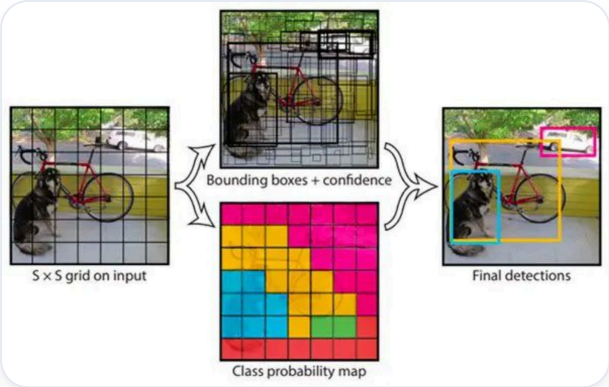
**NMS: quitar repetidas**

celdas vecinas ven el mismo coche y salen cajas duplicadas

- Sin etapa de propuestas: tiempo real desde la primera versión (2016).
- Varias celdas ven el mismo objeto: hace falta NMS al final.
- Hay muchas versiones, de grupos distintos; las de Ultralytics tienen licencia AGPL.

# YOLO: detección como regresión directa

YOLO (You Only Look Once) reformula la detección como un **problema de regresión único**. La imagen se divide en una cuadrícula S×S, donde cada celda predice directamente las cajas delimitadoras y las probabilidades de clase de los objetos cuyo centro cae en ella.




**La ventaja:** una única pasada hacia adelante por la red permite velocidades de procesamiento en tiempo real (30+ FPS), manteniendo precisión competitiva. Esta arquitectura one-stage elimina la necesidad de generar propuestas de región por separado.

- Cada celda predice múltiples cajas con sus scores de confianza.
- Las predicciones ocurren simultáneamente para toda la imagen.
- Ideal para aplicaciones que requieren latencia mínima.

# Detectores de una etapa

Comprueba en qué se diferencia un detector de una etapa.

## ¿Qué describe la idea de un detector de una etapa?

- A Propone regiones y las clasifica en un segundo recorrido.
- B No necesita imágenes de entrenamiento.
- C Predice localización y categoría en una sola pasada de la red.

Respuesta C. Exacto: esa es la idea central, aunque los detalles cambian entre versiones.

Un detector de una etapa predice cajas y clases en una única pasada sobre toda la imagen, sin una fase previa que proponga regiones candidatas.

# Evolución de la familia YOLO (2016-2026)

Diez años de versiones publicadas por grupos distintos. El número no indica quién hizo cada modelo ni que herede el código del anterior.

### 2016 · YOLOv1

Detección en tiempo real con una sola pasada de la red. Mucho más rápido que los detectores de dos etapas, aunque menos preciso.

### 2018 · YOLOv3

Predicción a tres escalas y backbone Darknet-53, con mejor detección de objetos pequeños.

### 2020 · YOLOv4

Backbone CSPDarknet53 y trucos de entrenamiento ("Bag of Freebies", como el aumento Mosaic) que mejoran la precisión sin frenar la inferencia.

### 2020 · YOLOv5

Primera versión en PyTorch, de Ultralytics. Popularizó YOLO por lo fácil que era entrenarlo y exportarlo.

### 2022 · YOLOv7

Arquitectura E-ELAN. Superó en velocidad y precisión a los detectores conocidos entre 5 y 160 FPS.

### 2023 · YOLOv8

De Ultralytics. Pasó a una cabeza sin anchors y unificó en una API detección, segmentación, pose y clasificación.

### 2024 · YOLOv9

PGI y GELAN para que la información útil no se pierda en redes profundas. La rama auxiliar se quita al inferir y no añade coste.

### 2024 · YOLOv10

De la Universidad de Tsinghua. Prescinde del NMS al inferir.

### 2024 · YOLO11

Sucesor de YOLOv8 en Ultralytics, sin la "v" en el nombre: más precisión con menos parámetros.

### 2025 · YOLO12

Versión comunitaria centrada en la atención: introduce atención por áreas sin perder el tiempo real.

### 2026 · YOLO26

Última de Ultralytics (enero). Cabeza uno a uno opcional sin NMS y regresión sin DFL, pensada para edge.

# Límites de las CNN

Por qué se pasó de las CNN a los transformers en detección.

### Miran por zonas

Cada filtro ve un trozo pequeño de la imagen. Para relacionar cosas alejadas hacen falta muchas capas.

### Piezas a mano

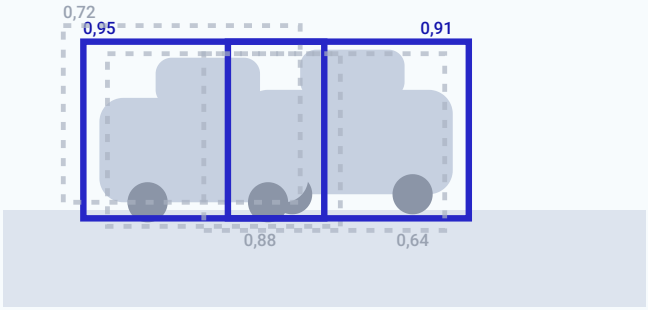
Los detectores con CNN suelen necesitar NMS y otros ajustes que se eligen a mano.

### Lo que cambia con transformers

La atención relaciona toda la imagen desde el principio, y DETR prescinde del NMS.

# NMS: el umbral de solapamiento

El detector propone varias cajas por coche. El NMS se queda con la de mayor puntuación y borra las que se solapan con ella por encima del umbral. Baja el umbral y desaparece un coche entero; súbelo y vuelven los duplicados.




Cinco cajas candidatas sobre dos coches. El detector las propone todas antes de filtrar.

Cajas que quedan

**2**

Suprimidas

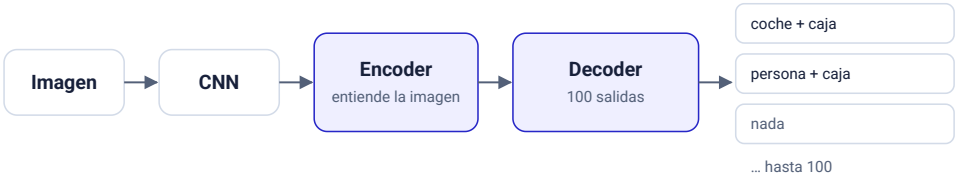
**3**

Una caja por coche: el umbral limpia los duplicados sin borrar objetos.

Esta página recoge el estado con umbral 0,50. El algoritmo recorre las cajas de mayor a menor puntuación: cada superviviente suprime a las que solapan con ella por encima del umbral. Con 0,20 la caja del primer coche llega a suprimir la del segundo, porque su IoU es 0,25, y se pierde un objeto real. Con 0,80 no se suprime nada y quedan las cinco cajas. No hay un valor universalmente correcto: depende de cuánto se solapen los objetos del dominio, y por eso DETR y RF-DETR se entrenan para no necesitarlo.

# DETR: detección con transformers

Devuelve directamente una lista fija de objetos. Lo que sobra dice "aquí no hay nada".

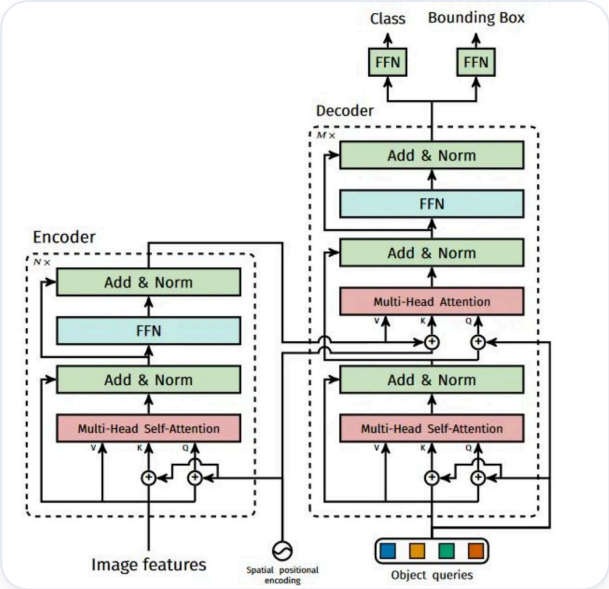



- **Encoder:** cada zona de la imagen tiene en cuenta toda la imagen.
- **Decoder:** devuelve 100 salidas; cada una es un objeto o "nada".
- Sin anchors ni NMS: cada objeto sale una sola vez.

# Esquema de DETR

DETR (Facebook, 2020) usa un transformer y devuelve directamente la lista de objetos.

- Una CNN saca las características.
- El encoder relaciona todas las zonas de la imagen.
- El decoder devuelve un número fijo de salidas, 100: cada una es un objeto con su caja o "nada".
- Sin anchors ni NMS.



# RF-DETR

RF-DETR (Roboflow, 2025; publicado en ICLR 2026) es un detector en tiempo real de la familia DETR. Su versión 2XL fue el primer detector en tiempo real en pasar de 60 AP en COCO, y la nano da 48,0 AP, 5,3 más que D-FINE nano con una latencia parecida.

### Backbone DINOv2 preentrenado

Parte de un vision transformer entrenado sin etiquetas sobre millones de imágenes (tema 4). Por eso se adapta rápido a un dominio nuevo con pocos datos.

### Atención deformable en el decoder

En lugar de mirar todos los píxeles, el decoder consulta unos pocos puntos de muestreo en cada escala. Así trabaja con mapas de alta resolución sin disparar el coste, lo que ayuda con los objetos pequeños.

### Sin anchors ni NMS, en varios tamaños

Como todo DETR, no necesita anchors predefinidos ni supresión de no máximos. Una búsqueda de arquitectura (NAS) entrena una sola red y saca de ella versiones de nano a 2XL sin reentrenar.

# Ultralytics y su licencia

Ultralytics es muy popular, pero este curso necesita código abierto que se pueda leer por dentro y con licencias que no limiten ni la investigación ni el uso industrial.

### Licencia problemática

Código y pesos van con AGPL-3.0, que obliga a publicar el código de la aplicación que los incorpora cuando la distribuyes, y también cuando solo la ofreces como servicio en red. Para evitarlo hay que pagar su licencia comercial. AGPL es open source, pero choca con la mayoría de usos industriales.

### Apropiación de marca YOLO

Sus versiones (YOLOv5, YOLOv8, YOLO11 y YOLO26) se presentan como la continuación de YOLO, pero no vienen de Joseph Redmon ni de la línea académica que siguió su trabajo. Como otros grupos numeran también sus YOLO (v6, v7, v9, v10, v12), el número ya no dice de dónde sale cada modelo.

### Código no modular

Está pensado para usarse a través de su CLI y de su API de alto nivel. Estudiarlo por dentro o adaptarlo a un proyecto de investigación cuesta bastante más.

### Limitaciones académicas

Varias versiones salieron sin un paper que describa la arquitectura y los experimentos, lo que dificulta reproducir resultados y explicarlos en clase.

Las condiciones de licencia se revisan por versión y por artefacto: el código y los pesos pueden tener términos distintos y cambiar entre versiones. Conviene comprobar la licencia vigente del repositorio y del checkpoint concreto antes de decidir.

# Por qué RF-DETR en el curso

RF-DETR es el detector de referencia del curso: su arquitectura es moderna y legible, y el proyecto es open source con licencia Apache 2.0.

### Licencia Apache 2.0

Permite usarlo, modificarlo y venderlo en industria, investigación y enseñanza, sin obligación de publicar tu código.

### Arquitectura moderna y clara

Es un DETR: backbone, decoder y salida directa, sin anchors ni NMS. Entenderlo es entender cómo se construyen los detectores actuales.

### Código limpio y modular

Se lee y se modifica bien, y el mismo paquete sirve para inferir y para reentrenar con tus datos.

### Velocidad y precisión

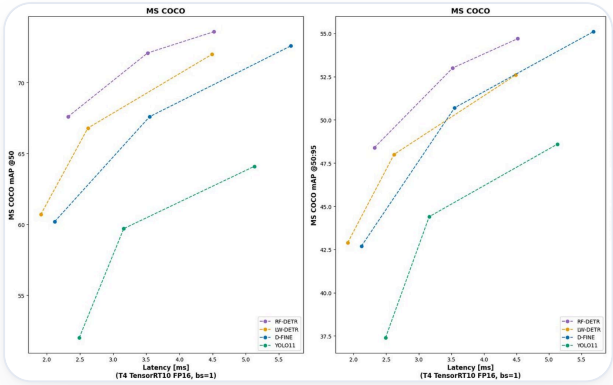
A igual latencia en una T4, da más mAP en COCO que YOLO11 según las curvas de su repositorio (siguiente pantalla).

# RF-DETR vs YOLO en producción

La elección entre RF-DETR y YOLO depende de tus **prioridades específicas de producción**.

**Consideraciones prácticas:**

- **Licencia:** los YOLO de Ultralytics usan AGPL-3.0, que obliga a publicar el código de la aplicación que los incorpora al distribuirla o servirla; RF-DETR usa Apache 2.0, que no lo exige.
- **Mantenibilidad:** RF-DETR evita el ajuste manual de anchors y umbrales de NMS.
- **Adopción:** YOLO tiene más tutoriales, integraciones y formatos de exportación probados, y más gente que ya lo conoce.




Latencia frente a mAP en MS COCO, medida en T4 con TensorRT10 y FP16.

Las curvas proceden del repositorio de RF-DETR y se miden en un hardware concreto. Antes de decidir conviene reproducir la medida en el dispositivo de destino, con el tamaño de entrada, la precisión numérica y el tamaño de lote que se vayan a usar realmente.

# Tiempo real

Un vídeo normal va a 25 o 30 fotogramas por segundo (FPS). Para seguirlo en directo, todo el trabajo de un fotograma tiene que caber en 40 milisegundos a 25 FPS, o en 33 a 30 FPS.

### Medir bien

FPS es 1 entre el tiempo por fotograma. Se mide con un fotograma cada vez, tras unas pasadas de calentamiento y contando todo: leer, preparar la imagen, el modelo, el posproceso y el dibujo.

### Ganar velocidad sin reentrenar

- Un modelo más pequeño: RF-DETR tiene versiones nano, small, medium, base y large.
- Menos resolución de entrada y media precisión (FP16).
- Detectar uno de cada N fotogramas y dejar que el seguidor cubra los demás.

### Llevarlo a producción

Exportar el modelo a ONNX y optimizarlo para el hardware: TensorRT en GPU de NVIDIA, OpenVINO en procesadores de Intel. Así están medidas las curvas de la pantalla anterior.

### Equipos pequeños

Junto a la cámara, en un NVIDIA Jetson por ejemplo, se usan las versiones pequeñas y la exportación optimizada. La cifra que vale es la medida en ese equipo, no la del portátil.

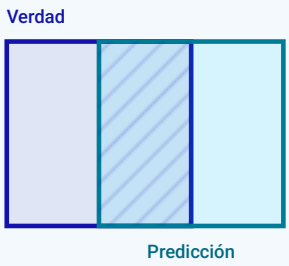
En el ejercicio de vídeo del cuaderno 1, RF-DETR medium procesa 200 fotogramas de 960 × 540 a unos 25 FPS en una RTX 3080 Ti de portátil, contando el proceso entero; una T4 de Colab va más despacio. Las versiones small y nano son más rápidas y algo menos precisas. Para decidir, se mide en el dispositivo de destino con la resolución, la precisión numérica y el tamaño de lote reales.

# IoU (Intersection over Union)

La **Intersection over Union (IoU)** mide cuánto se solapan dos cajas delimitadoras: la que predice el modelo y la caja de verdad (ground truth) anotada a mano. Es lo que decide si una predicción cuenta como acierto (True Positive) o como falsa alarma (False Positive). Va de 0, sin solapamiento, a 1, cajas idénticas.

En la figura, la caja predicha está en rojo y la de verdad en verde. La IoU divide el área donde se superponen entre el área total que cubren las dos juntas.

$$IOU = \frac{\text{area of overlap}}{\text{area of union}} = \frac{\square}{\square}$$




IoU

**0,33**

Por debajo de 0,5: contaría como falsa alarma (FP).

### Umbrales típicos de IoU

- **IoU ≥ 0,5:** el umbral clásico de PASCAL VOC para contar un acierto.
- **IoU ≥ 0,75:** más estricto, exige cajas casi exactas (AP75).
- **IoU por debajo del umbral:** la detección cuenta como False Positive (FP).

### ¿Por qué es importante?

- Sin IoU no habría forma de separar los aciertos (TP) de las falsas alarmas (FP).
- Es la base de las métricas que vienen después: precisión, recall, Average Precision (AP) y mean Average Precision (mAP).
- Mide la exactitud espacial de cada detección, además de si acierta la clase.

IoU vale 1 cuando ambas cajas coinciden y 0 cuando no comparten área. Para contar un acierto también tiene que coincidir la clase, y cada objeto real solo puede dar un acierto.

# TP, FP y FN (no hay TN)

**TP (True positives)**

Detecciones correctas: cajas predichas que coinciden con objetos reales (IoU > umbral).

**FP (False positives)**

Falsas alarmas: cajas predichas donde no hay objeto real o con IoU por debajo del umbral.

**FN (False negatives)**

Objetos perdidos: objetos reales que el modelo no detectó.

## Por qué NO existen True Negatives (TN) en detección de objetos

La métrica de True Negatives, común en problemas de clasificación binaria, no aplica de la misma manera en la detección de objetos:

- **En clasificación binaria:** TN = "predice que no hay gato y efectivamente no hay gato".
- **En detección de objetos:** el modelo predice cajas delimitadoras en ubicaciones específicas donde cree que hay un objeto.
- No existe el concepto explícito de "predecir ausencia de objeto en una ubicación".
- Los sitios donde no se predice nada son infinitos: no se pueden contar.

Por esta razón, las métricas en detección de objetos se enfocan únicamente en **TP, FP y FN**, que son los únicos resultados que se pueden contar.

# Precisión y recall

Saber si el modelo ha detectado algo no basta. La precisión dice cuánto puedes fiarte de lo que detecta; el recall, cuánto de lo que hay consigue encontrar.

### La importancia de la precisión

Mide la **fiabilidad de las detecciones**: con alta precisión hay pocos falsos positivos (detectar algo que no está o confundir su clase). Importa cuando una falsa alarma sale cara, como al contar inventario o cuando cada aviso obliga a parar una línea.

$$\text{Precisión} = \frac{TP}{TP + FP}$$

Responde a: "De todas mis predicciones, ¿cuántas son correctas?".

### La importancia del recall

Mide la **capacidad de encontrar todos los objetos**: con alto recall hay pocos falsos negativos (objetos presentes que no se detectan). Importa cuando perder uno es grave, como una persona en vigilancia o un tumor en imagen médica.

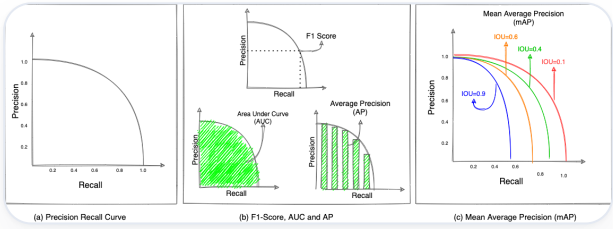
$$\text{Recall} = \frac{TP}{TP + FN}$$

Responde a: "De todos los objetos reales, ¿cuántos he detectado?".

**Cómo se leen:**

- **Precisión 0,9:** el 90 % de las detecciones son correctas. Puedes fiarte de lo que dice haber encontrado.
- **Recall 0,8:** ha encontrado el 80 % de los objetos que había. Se le escapa uno de cada cinco.

Las dos suelen tirar en direcciones opuestas, un **trade-off**, aunque la precisión no siempre sube al filtrar. Un modelo que detecta con mucha manga ancha para no perderse nada (recall alto) también da más falsas alarmas (precisión baja). El punto de equilibrio lo decide la aplicación.



# Calcula precisión y recall

Aplica las dos fórmulas a un recuento concreto.

## Hay 3 TP, 1 FP y 2 FN. ¿Cuáles son precisión y recall?

- A Precisión 0,75; recall 0,60.
- B Precisión 0,60; recall 0,75.
- C Precisión 0,75; recall 0,75.

Respuesta A. Exacto: precisión 3/(3+1) y recall 3/(3+2).

Con 3 TP, 1 FP y 2 FN: precisión = 3 / (3 + 1) = 0,75 y recall = 3 / (3 + 2) = 0,60. La precisión mira el denominador de las predicciones; el recall, el de los objetos reales.

# F1: precisión y recall en un solo número

Para comparar dos umbrales o dos modelos a veces hace falta una sola cifra. La F1 junta precisión y recall, y baja en cuanto una de las dos es mala.

### Cómo se calcula

$$F1 = \frac{2 \cdot \text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}}$$

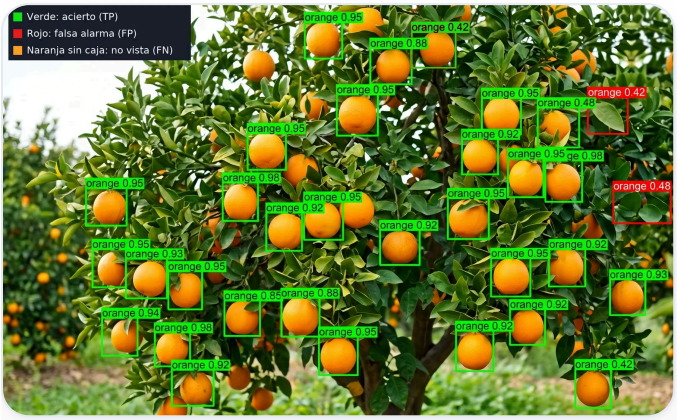
Es la media armónica de las dos. Con el recuento de la pantalla anterior, precisión 0,75 y recall 0,60, da **F1 = 0,67**.

### Por qué no la media de siempre

Un detector que marca un solo objeto, y acierta, tiene precisión 1,0 y recall 0,1. La media normal le daría 0,55. La F1 le da **0,18**: si una de las dos falla, la F1 también.

- **Para elegir el umbral:** el que da la F1 más alta es un buen punto de partida cuando los dos errores cuestan parecido.
- **Si un error cuesta más:** la F-beta da más peso a uno de los dos. La F2, por ejemplo, prima el recall.
- **No sustituye a la AP:** la F1 mira un solo umbral; la AP resume todos.

# Aciertos, falsas alarmas y objetos perdidos




- **Verde, TP:** 31 cajas sobre naranjas de verdad.
- **Rojo, FP:** 2 cajas sobre hojas, con 0,42 y 0,48 de confianza.
- **Sin caja, FN:** naranjas que no ha visto, como la de arriba a la izquierda o las tapadas por hojas.
- **TN:** no se cuentan. ¿Cuántas "no naranjas" hay en la foto?

Precisión: 31 de 33 cajas, 0,94. Para el recall hay que contar a mano todas las naranjas, y eso es anotar.

Imagen generada con IA para el curso, con las cajas dibujadas como las daría un detector. Las dos falsas alarmas tienen confianzas bajas: un umbral de 0,50 las quitaría, pero también quitaría la naranja de abajo a la derecha, que tiene 0,42.

# Umbral, precisión y recall

Sube el umbral y mira las dos cifras. No hay ningún punto en el que las dos valgan 1: toca elegir.




Precisión

**0,60**

Recall

**1,00**

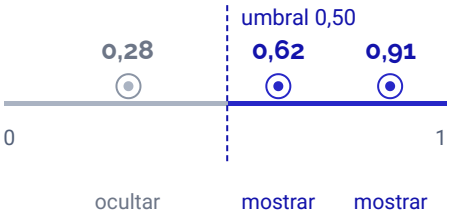
TP 3 FP 2 FN 0

Se ven todos los objetos, pero también sobran predicciones.

Esta página recoge el estado con umbral 0,30: tres aciertos y dos falsas alarmas (0,66 y 0,41), precisión 0,60 y recall 1. Con 0,50 se va la de 0,41: precisión 0,75, recall 1. Con 0,60 se pierde el objeto de 0,55: 0,67 y 0,67. Con 0,70 ya no queda ninguna falsa alarma, pero el recall es 0,67. La falsa alarma de 0,66 puntúa más que un acierto, así que ningún umbral deja las dos cifras en 1.

# Umbral de confianza e IoU

El umbral de confianza decide qué predicciones se muestran; IoU evalúa su solapamiento.




El filtro cambia qué predicciones ves, no sus coordenadas.

**Subes el umbral de confianza sin cambiar las predicciones candidatas. ¿Qué pasa con las cajas que se muestran?**

- A Se conservan las mismas o menos cajas.
- B Aparecen categorías nuevas.
- C Aumenta necesariamente la IoU de todas las cajas restantes.

Respuesta A. Exacto: el filtro elimina las puntuaciones que quedan por debajo del nuevo umbral.

Al subir el umbral suelen desaparecer predicciones de baja puntuación: pueden bajar los falsos positivos y también aumentar los falsos negativos. La curva precisión-recall estudia ese intercambio recorriendo muchos umbrales.

# Qué error cuesta más

El umbral depende de qué error sale más caro en cada aplicación.

**¿Para qué usas el detector?**

- (●) Seguridad: detectar a una persona en zona restringida
- ( ) Inspección: encontrar piezas defectuosas antes de enviarlas
- ( ) Inventario: contar producto en una estantería
- ( ) Analítica: estimar flujo de gente en una calle

### Prioriza el recall

No detectar a alguien que sí está es el fallo grave. Una falsa alarma cuesta una comprobación; un objeto perdido cuesta el incidente.

**Qué hacer:** baja el umbral de confianza y asume más falsos positivos. Añade una segunda comprobación, humana o automática, para filtrarlos.

### Prioriza el recall, con revisión humana

Dejar pasar una pieza defectuosa llega al cliente; marcar una buena solo cuesta una revisión en planta.

**Qué hacer:** umbral bajo y una estación de revisión. Mide cuántas piezas buenas estás parando, porque ese es el coste que notará producción.

### Prioriza la precisión

Contar de más esconde las roturas de stock: el sistema cree que hay producto y no lo repone a tiempo.

**Qué hacer:** sube el umbral hasta que apenas haya falsas alarmas y vigila el recuento frente a un conteo manual de control.

### Busca el equilibrio y mide la tendencia

Aquí no importa cada persona concreta sino que el sesgo sea estable. Un error constante se corrige; uno que cambia con la hora o el clima, no.

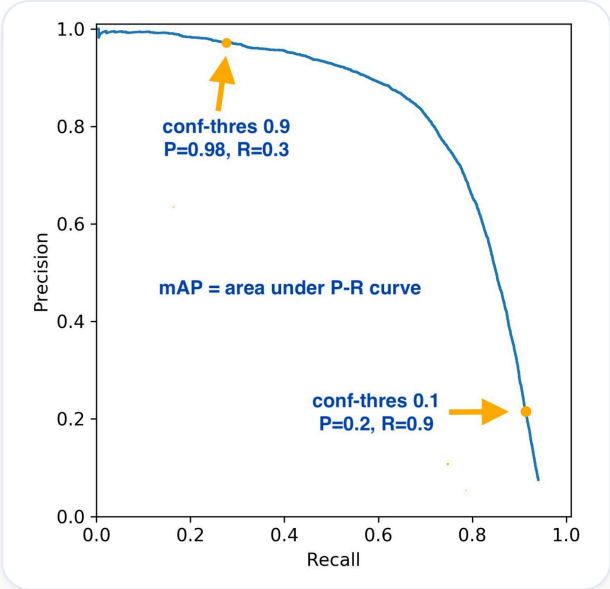
**Qué hacer:** deja el umbral intermedio y valida sobre grabaciones de momentos distintos, no sobre una sola.

La regla general: si el coste de perder un objeto supera al de revisar una falsa alarma, baja el umbral; si es al revés, súbelo. Y en todos los casos, mide el efecto sobre tus propios datos, no sobre el benchmark.

# AP y mAP

**AP (Average Precision):** es el área bajo la curva precisión-recall. Resume en un número lo bien que el modelo encuentra los objetos (recall alto) sin llenarse de falsos positivos (precisión alta). Cuanto más alta, mejor.

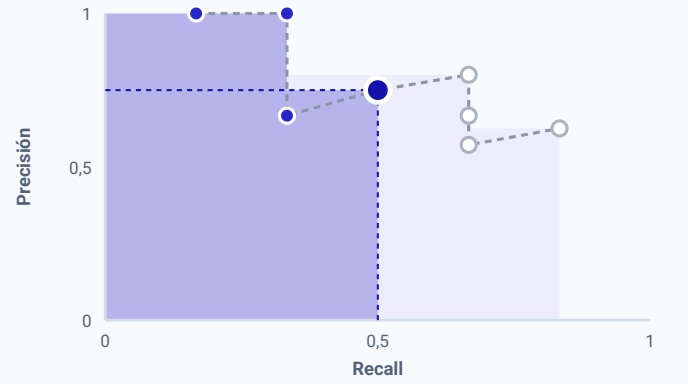
**mAP (mean Average Precision):** la métrica estándar para comparar detectores. Es la media de la AP de todas las clases del conjunto de evaluación. mAP@0.5 la calcula con IoU 0,5; mAP@[0.5:0.95], la métrica principal de COCO, la promedia además entre diez umbrales de 0,50 a 0,95.




Recorrer el umbral de confianza traza la curva: con umbral 0,9 la precisión es alta y el recall bajo; con 0,1 ocurre lo contrario. AP50 y AP promediada entre varios umbrales de IoU responden a protocolos distintos y no son intercambiables.

# Curva precisión-recall y AP

Ocho detecciones de una clase, de más a menos confianza, para seis objetos reales. Baja el umbral: cada detección que entra añade un punto a la curva. La AP es el área bajo la curva.




| 0,95 | 0,90 | 0,82 | 0,76 | 0,64 | 0,51 | 0,43 | 0,30 |
|---|---|---|---|---|---|---|---|
| TP | TP | FP | TP | TP | FP | FP | TP |

Precisión

**0,75**

Recall

**0,50**

AP, el área bajo la curva: **0,70**

Con umbral 0,76 entra un acierto: el recall sube a 0,50 y la precisión queda en 0,75.

Esta página recoge el estado con umbral 0,76: cuatro detecciones aceptadas, tres aciertos y un falso positivo, con precisión 0,75 y recall 0,50. Cada acierto sube el recall un sexto; cada falso positivo baja la precisión sin mover el recall. Para el área, la curva se convierte en escalera: cada punto toma la mejor precisión que se alcanza a ese recall o a uno mayor. Con las ocho detecciones el recall llega a 0,83, porque un objeto no lo encuentra ninguna, y la AP vale 0,70.

# Métricas operativas

La mAP sirve para comparar modelos. Para ponerlos en marcha se fija un umbral, el punto de trabajo, y se mide lo que pasará con él.

### Recall a precisión fija

"Recall al 95 % de precisión" (R@P95): cuánto encuentra el modelo si solo admites un 5 % de falsas alarmas entre lo que marca.

### Falsos positivos por imagen

FPPI, o falsas alarmas por hora de vídeo: la cifra que entiende quien recibe los avisos. Se da junto al recall, por ejemplo "recall a 0,1 FPPI".

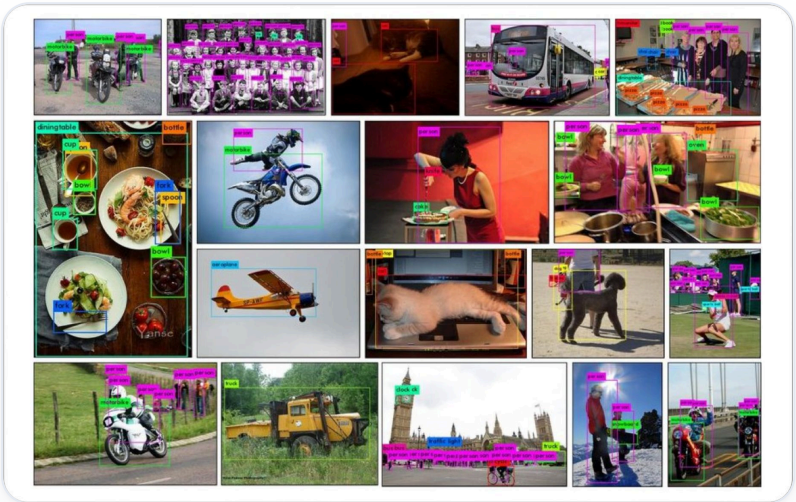
### TPR a un TNR del 95 %

En clasificación y anomalías, donde sí hay negativos que contar: aceptando el 95 % de las piezas buenas, qué parte de las defectuosas detectas. Vuelve en el tema 4.

En detección no hay verdaderos negativos (pantalla "Errores"), por eso se habla de falsos positivos por imagen y no de TNR. El punto de trabajo se elige con datos de validación y se vigila después en producción.

# Límites de COCO

- **COCO:** 80 clases de objetos cotidianos (persona, coche, perro, silla…). Sirve para comparar modelos.
- **En una fábrica** hacen falta otras clases: "tornillo oxidado", "conector suelto", "soldadura defectuosa". No están en COCO.
- **Adaptar el modelo** exige anotar imágenes nuevas y entrenar: tiempo de anotación y de GPU.
- **Alternativa:** detectores a los que se les escribe la clase en texto. Se ven en el tema 3.



# Las clases salen de los datos

Un checkpoint aprende las clases de sus datos. Tu aplicación puede necesitar otras.

**Tu modelo solo conoce las clases de COCO y necesitas detectar un defecto nuevo.**

- A Bajar el umbral hasta que aparezca el defecto.
- B Entrenar con imágenes etiquetadas del defecto, o probar un detector que acepte la clase escrita.
- C Usar el mismo modelo: entre 80 clases alguna se parecerá.

Respuesta B. Exacto: una clase nueva pide datos propios o un modelo de vocabulario abierto, como los del tema 3.

Un detector de vocabulario cerrado solo puede nombrar las categorías con las que fue entrenado. Ampliarlo exige anotar y reentrenar, o cambiar a un enfoque guiado por texto.

# Cómo evaluar un detector

Además de la mAP en COCO:

**1. Medir en tu dominio específico:** crea un conjunto de validación representativo de tus datos reales. Un modelo con 60 % mAP en COCO puede tener 40 % o 80 % en tu caso de uso particular.

**2. Equilibrar velocidad y precisión:** un detector con 2 % más de AP pero un 50 % más lento puede no compensar si necesitas procesar 30 FPS. Mide la latencia de extremo a extremo, no la cifra teórica del paper.

**3. Analizar errores cualitativamente:** inspecciona visualmente los casos de:

- **Falsos positivos (FP):** detecciones donde no hay objeto.
- **Falsos negativos (FN):** objetos que el modelo no detectó.
- **Casos límite:** objetos parcialmente ocluidos, muy pequeños, o en condiciones de iluminación extrema.

Los patrones de error te dirán si necesitas más datos, data augmentation específico, o cambiar de arquitectura.

# Dónde medir

Un benchmark no sustituye datos representativos, latencia medida y análisis de errores.

## Un detector logra mejor AP en un benchmark publicado. ¿Basta para elegirlo?

- A Sí, cualquier mejora de AP garantiza mejor producción.
- B Sí, si es el modelo más reciente.
- C No: hay que medir calidad y latencia con datos y condiciones representativos.

Respuesta C. Exacto: la elección depende del problema, del entorno y de los errores que aceptas.

Un AP mejor en un benchmark público indica rendimiento sobre ese conjunto y ese protocolo. La decisión necesita además datos de tu dominio, latencia medida en el hardware de destino y revisión de los errores que importan.

# Coste de anotar un dataset

Mueve los tres controles y mira cuántas horas cuesta anotar tus propias clases antes de entrenar.

Clases que necesitas **4**

Imágenes por clase **300**

Segundos por imagen **45**

Imágenes que anotar

**1200**

Horas, con revisión incluida

**20**

Jornadas de una persona

**3,3**

Coste asumible para un conjunto propio. Aun así, empieza probando un detector guiado por texto (tema 3) para saber si hace falta.

Estimación de orden de magnitud. Suma un 30 % de revisión sobre el tiempo de anotación y cuenta 6 horas productivas por jornada.

La página recoge el caso de 4 clases, 300 imágenes por clase y 45 segundos por imagen. Las cifras que más se disparan son las clases y los segundos por imagen: anotar segmentación o puntos clave multiplica el tiempo respecto a poner cajas. Antes de asumir ese coste conviene comprobar cuánto resuelve un detector guiado por texto sin anotar nada, que se ve en el tema 3.

# Antes de poner un detector en producción

Ocho comprobaciones antes de ponerlo en marcha.

- [ ] **Conjunto de validación propio**
  Imágenes de tu dominio, tus cámaras y tus condiciones de luz. No basta el mAP del benchmark.
- [ ] **Umbral elegido con criterio**
  Decidido según qué error cuesta más en tu caso, no dejado en el valor por defecto.
- [ ] **Latencia medida en el dispositivo real**
  End to end, incluyendo carga, preprocesado y posprocesado. No el número del paper.
- [ ] **Revisión visual de los errores**
  Mira falsos positivos, falsos negativos y casos límite: ocluidos, pequeños, contraluz.
- [ ] **Licencia comprobada**
  Del repositorio y de los pesos, en la versión concreta que vas a desplegar.
- [ ] **Versión y revisión fijadas**
  Anota el identificador del modelo y su revisión para poder reproducir el resultado.
- [ ] **Comportamiento ante entradas raras**
  Imagen vacía, muy oscura, resolución distinta, formato inesperado.
- [ ] **Plan de seguimiento**
  Cómo vas a detectar que el rendimiento cae cuando cambien las condiciones.

# Cuaderno 1: detección de objetos

Los ejercicios están en el cuaderno. El extra, si da tiempo, lo hacemos juntos; si no, queda para casa.

1. Detectar objetos con RF-DETR
2. Cambiar el umbral de confianza y contar
3. Ejercicio: tu propia imagen
4. Ajuste fino: jugadores, árbitros y balón de baloncesto
5. Extra: quedarse solo con algunas clases

**Abrir en Colab ↗**

En Colab: Archivo, Guardar una copia en Drive, y Entorno de ejecución, GPU T4.